

K8S persistent volumes

Вступ

Керування сховищем є відмінною проблемою від керування обчислювальними екземплярами. Підсистема PersistentVolume надає API для користувачів і адміністраторів, який абстрагує деталі того, як надається сховище, від того, як воно споживається. Для цього ми представляємо два нові ресурси API: PersistentVolume та PersistentVolumeClaim.

PersistentVolume (PV) — це частина сховища в кластері, яка була надана адміністратором або динамічно надана за допомогою класів сховища. Це ресурс у кластері так само, як вузол є ресурсом кластера. PV — це плагіни томів, як Volumes, але мають життєвий цикл, незалежний від будь-якого окремого Pod, який використовує PV. Цей об'єкт API фіксує деталі реалізації сховища, будь то NFS, iSCSI або система зберігання, яка відповідає конкретному хмарному провайдеру.

Життєвий цикл volume та claim

PV - це ресурси в кластері. PVC є запитом на ці ресурси, а також виконують функцію перевірки претензій до ресурсу. Взаємодія між PV і PVC відповідає такому життєвому циклу:

Забезпечення

Існує два способи надання PV: статичний або динамічний.

Статичний

Адміністратор кластера створює кілька PV. Вони містять деталі реального сховища, яке доступне для використання користувачами кластера. Вони існують в API Kubernetes і доступні для використання.

ЖИТТЄВИЙ цикл volume та claim

Динамічний

Якщо жодна зі статичних PV, створених адміністратором, не відповідає PersistentVolumeClaim користувача, кластер може спробувати динамічно надати том спеціально для PVC. Це надання базується на StorageClasses: PVC має запитувати клас зберігання, і адміністратор повинен створити та налаштувати цей клас для динамічного надання. Твердження, які запитують клас "", фактично вимикають динамічне надання для себе.

Щоб увімкнути динамічне надання сховища на основі класу сховища, адміністратору кластера потрібно увімкнути контролер доступу `DefaultStorageClass` на сервері API. Це можна зробити, наприклад, переконавшись, що `DefaultStorageClass` є серед розділених комами впорядкованого списку значень для прапорця `--enable-admission-plugins` компонента сервера API.

Життєвий цикл volume та claim

Прив'язка

Користувач створює або, у разі динамічного надання, вже створив PersistentVolumeClaim із певним обсягом запитаної пам'яті та з певними режимами доступу. Цикл керування в головному спостерігає за новими PVC, знаходить відповідний PV (якщо можливо) і зв'язує їх разом. Якщо PV було динамічно підготовлено для нового PVC, цикл завжди прив'язуватиме цей PV до PVC. В іншому випадку користувач завжди отримає принаймні те, що просив, але обсяг може перевищити запитаний. Після прив'язки прив'язки PersistentVolumeClaim є ексклюзивними, незалежно від того, як вони були прив'язані. Прив'язка PVC до PV – це відображення один-до-одного з використанням ClaimRef, який є двонаправленим зв'язуванням між PersistentVolume та PersistentVolumeClaim.

Претензії залишатимуться необмеженими на невизначений термін, якщо відповідний том не існує. Претензії будуть пов'язані, коли будуть доступні відповідні томи. Наприклад, кластер із багатьма PV 50Gi не відповідатиме PVC, який запитує 100Gi. PVC можна зв'язати, коли до кластера додається PV 100Gi.

Життєвий цикл volume та claim

Використання

Поди використовують претензії як томи. Кластер перевіряє претензію, щоб знайти зв'язаний том і монтує цей том для модуля. Для томів, які підтримують кілька режимів доступу, користувач визначає, який режим є потрібним, коли використовує свою вимогу як том у модулі.

Коли користувач має претензію, і ця претензія зв'язана, зв'язана PV належить користувачеві доти, доки вона йому потрібна. Користувачі планують модулі та отримують доступ до своїх заявлених PV, включивши розділ `persistentVolumeClaim` до блоку томів модуля.

Життєвий цикл volume та claim

Об'єкт зберігання в режимі захисту від використання

Метою функції захисту об'єктів зберігання під час використання є гарантія того, що PersistentVolumeClaims (PVC), які активно використовуються модулем, і PersistentVolume (PV), прив'язані до PVC, не видаляються із системи, оскільки це може призвести до втрати даних.

Примітка: PVC активно використовується модулем, якщо існує об'єкт Pod, який використовує PVC.

Якщо користувач видаляє PVC, який активно використовується модулем, PVC не видаляється негайно. Видалення PVC відкладено, доки PVC більше не буде активно використовуватися будь-якими контейнерами. Крім того, якщо адміністратор видаляє PV, який прив'язаний до PVC, PV не видаляється негайно. Видалення PV відкладається до тих пір, поки PV більше не буде зв'язано з PVC.

ЖИТТЄВИЙ цикл volume та claim

Об'єкт зберігання в режимі захисту від використання

Ви можете побачити, що PVC захищено, якщо статус PVC — Термінування, а список фіналізаторів містить `kubernetes.io/pvc-protection` :

```
kubectl describe pvc hostpath
Name:          hostpath
Namespace:     default
StorageClass:  example-hostpath
Status:        Terminating
Volume:
Labels:        <none>
Annotations:   volume.beta.kubernetes.io/storage-class=example-hostpath
               volume.beta.kubernetes.io/storage-provisioner=example.com/hostpath
Finalizers:    [kubernetes.io/pvc-protection]
...
```


ЖИТТЄВИЙ цикл volume та claim

Ви можете побачити, що PV захищено, коли статус PV має статус `Terminating`, а список `Finalizers` також містить `kubernetes.io/pv-protection` :

```
kubectl describe pv task-pv-volume
Name:          task-pv-volume
Labels:        type=local
Annotations:   <none>
Finalizers:    [kubernetes.io/pv-protection]
StorageClass:  standard
Status:        Terminating
Claim:
Reclaim Policy: Delete
Access Modes:  RWO
Capacity:      1Gi
Message:
Source:
  Type:        HostPath (bare host directory volume)
  Path:        /tmp/data
  HostPathType:
Events:        <none>
```

Життєвий цикл volume та claim

Відновлення

Коли користувач закінчить роботу зі своїм томом, він може видалити об'єкти PVC з API, що дозволяє відновити ресурс. Політика повернення для PersistentVolume повідомляє кластеру, що робити з томом після того, як його претензію було звільнено. Наразі томи можна зберегти, переробити або видалити.

Життєвий цикл volume та claim

Збереження

Політика Retain reclaim дозволяє ручне відновлення ресурсу. Коли PersistentVolumeClaim видалено, PersistentVolume все ще існує, і том вважається «вивільненим». Але він ще не доступний для іншої претензії, оскільки дані попереднього заявника залишаються на томі. Адміністратор може вручну відновити том, виконавши такі дії.

1. Видаліть PersistentVolume. Пов'язаний ресурс зберігання в зовнішній інфраструктурі (наприклад, AWS EBS, GCE PD, диск Azure або том Cinder) все ще існує після видалення PV.
2. Відповідно очистіть дані в пов'язаному сховищі вручну.
3. Вручну видаліть пов'язаний ресурс зберігання або, якщо ви хочете повторно використовувати той самий ресурс зберігання, створіть новий PersistentVolume з визначенням ресурсу зберігання.

Життєвий цикл volume та claim

Видалення

Для плагінів томів, які підтримують політику відновлення видалення, видалення прибирає як об'єкт PersistentVolume з Kubernetes, так і пов'язаний ресурс зберігання в зовнішній інфраструктурі, як-от том AWS EBS, GCE PD, Azure Disk або Cinder. Томи, які були динамічно підготовлені, успадковують політику повернення свого класу зберігання, яка за умовчанням має значення Delete. Адміністратор повинен налаштувати StorageClass відповідно до очікувань користувачів; інакше PV потрібно відредагувати або виправити після його створення.

Попередження: політика Recycle Reclaim застаріла. Замість цього рекомендується використовувати динамічне надання.

Якщо підтримується плагіном основного тома, політика Recycle reclaim виконує базове очищення (`rm -rf /thevolume/*`) тома та знову робить його доступним для нової претензії.

ЖИТТЄВИЙ цикл volume та claim

Однак адміністратор може налаштувати спеціальний шаблон Recycler Pod за допомогою аргументів командного рядка менеджера контролера Kubernetes. Спеціальний шаблон Recycler Pod має містити специфікацію обсягів, як показано в прикладі нижче:

```
apiVersion: v1
kind: Pod
metadata:
  name: pv-recycler
  namespace: default
spec:
  restartPolicy: Never
  volumes:
  - name: vol
    hostPath:
      path: /any/path/it/will/be/replaced
  containers:
  - name: pv-recycler
    image: "k8s.gcr.io/busybox"
    command: ["/bin/sh", "-c", "test -e /scrub && rm -rf /scrub/..?* /scrub/.[!..]* /scrub/* && test -z \"$(ls -A /scrub)\" || exit 1"]
    volumeMounts:
    - name: vol
      mountPath: /scrub
```

ЖИТТЄВИЙ цикл volume та claim

Резервування PersistentVolume

Площина керування може прив'язувати PersistentVolumeClaims до відповідних PersistentVolumes у кластері. Однак, якщо ви хочете, щоб PVC зв'язувався з певним PV, вам потрібно попередньо зв'язати їх.

Вказуючи PersistentVolume у PersistentVolumeClaim, ви оголошуєте зв'язок між цим конкретним PV і PVC. Якщо PersistentVolume існує і не зарезервував PersistentVolumeClaims через своє поле claimRef, тоді PersistentVolume та PersistentVolumeClaim буде зв'язано.

Зв'язування відбувається незалежно від деяких критеріїв відповідності томів, включаючи спорідненість вузлів. Площина керування все ще перевіряє дійсність класу сховища, режимів доступу та запитаного розміру сховища.

```
apiVersion: v1
kind: PersistentVolumeClaim
metadata:
  name: foo-pvc
  namespace: foo
spec:
  storageClassName: "" # Empty string must be explicitly set otherwise default StorageClass will be set
  volumeName: foo-pv
...
```

Життєвий цикл volume та claim

Цей метод не гарантує жодних привілеїв прив'язки до PersistentVolume. Якщо інші PersistentVolumeClaims можуть використовувати вказаний вами PV, вам спочатку потрібно зарезервувати цей обсяг пам'яті. Укажіть відповідний PersistentVolumeClaim у полі claimRef PV, щоб інші PVC не могли зв'язуватися з ним.

Це корисно, якщо ви хочете споживати PersistentVolumes, у яких для політики вимоги встановлено значення Retain, включаючи випадки, коли ви повторно використовуєте існуючий PV.

```
apiVersion: v1
kind: PersistentVolume
metadata:
  name: foo-pv
spec:
  storageClassName: ""
  claimRef:
    name: foo-pvc
    namespace: foo
  ...
```

ЖИТТЄВИЙ цикл volume та claim

Розширення Persistent Volumes Claims

СТАН ФУНКЦІЇ: Kubernetes v1.11 [бета]

Підтримку розширення PersistentVolumeClaims (PVC) тепер увімкнено за замовчуванням. Ви можете розширити такі типи томів:

- gcePersistentDisk
- awsElasticBlockStore
- Cinder
- glusterfs
- rbd
- Azure File
- Azure Disk
- Portworx
- FlexVolumes
- CSI

```
apiVersion: storage.k8s.io/v1
kind: StorageClass
metadata:
  name: gluster-vol-default
provisioner: kubernetes.io/glusterfs
parameters:
  resturl: "http://192.168.10.100:8080"
  restuser: ""
  secretNamespace: ""
  secretName: ""
allowVolumeExpansion: true
```

You can only expand a PVC if its storage class's allowVolumeExpansion field is set to true.

Життєвий цикл volume та claim

Розширення обсягу CSI

СТАН ФУНКЦІЇ: Kubernetes v1.16 [бета]

Підтримку розширення томів CSI увімкнено за замовчуванням, але також потрібен спеціальний драйвер CSI для підтримки розширення томів. Для отримання додаткової інформації зверніться до документації конкретного драйвера CSI.

Зміна розміру тому, що містить файлову систему

Ви можете змінити розмір томів, що містять файлову систему, лише якщо це файлова система XFS, Ext3 або Ext4.

Якщо том містить файлову систему, розмір файлової системи змінюється лише тоді, коли новий Pod використовує PersistentVolumeClaim у режимі ReadWrite. Розширення файлової системи виконується або під час запуску Pod, або коли Pod працює, а базова файлова система підтримує розширення в режимі онлайн.

FlexVolumes дозволяють змінювати розмір, якщо для параметра RequiresFSResize для драйвера встановлено значення true. Розмір FlexVolume можна змінити після перезапуску Pod.

Життєвий цикл volume та claim

Зміна розміру використовуваного PersistentVolumeClaim

СТАН ФУНКЦІЇ: Kubernetes v1.15 [бета]

Примітка. Розширення поточних PVC доступне як бета-версія з Kubernetes 1.15 і як альфа-версія з 1.11. Необхідно ввімкнути функцію `ExpandInUsePersistentVolumes`, що відбувається автоматично для багатьох кластерів для бета-функцій.

У цьому випадку вам не потрібно видаляти та повторно створювати модуль або розгортання, які використовують існуючий PVC. Будь-який використовуваний PVC автоматично стає доступним для його Pod, щойно його файлову систему буде розширено. Ця функція не впливає на PVC, які не використовуються модулем або розгортанням. Ви повинні створити модуль, який використовує PVC, перш ніж завершиться розширення.

Подібно до інших типів томів, томи FlexVolume також можна розширити, коли вони використовуються за допомогою Pod.

Життєвий цикл volume та claim

Відновлення після збою під час розширення томів

Якщо розширити базове сховище не вдається, адміністратор кластера може вручну відновити стан вимоги постійного тому (PVC) і скасувати запити на зміну розміру. В іншому випадку запити на зміну розміру постійно повторюються контролером без втручання адміністратора.

1. Позначте PersistentVolume(PV), який прив'язано до PersistentVolumeClaim(PVC), політикою Retain reclaim.
2. Видаліть PVC. Оскільки для PV діє політика `Retain reclaim`, ми не втратимо жодних даних під час повторного створення PVC.
3. Видаліть запис `claimRef` зі специфікацій PV, щоб новий PVC міг прив'язуватися до нього. Це має зробити PV доступним.
4. Повторно створіть PVC із меншим розміром, ніж PV, і встановіть для поля `volumeName` PVC назву PV. Це повинно прив'язати новий ПВХ до існуючого ПВ.
5. Не забудьте відновити політику повернення PV.

Типи Persistent Volumes

Типи PersistentVolume реалізовані як плагіни. Наразі Kubernetes підтримує такі плагіни:

- `awsElasticBlockStore` - AWS Elastic Block Store (EBS)
- `azureDisk` - Azure Disk
- `azureFile` - Azure File
- `cephfs` - CephFS volume
- `cinder` - Cinder (блок зберігання OpenStack) (застарілий)
- `csi` - Container Storage Interface (CSI)
- `fc` - Fibre Channel (FC) storage
- `flexVolume` - FlexVolume
- `flocker` - Flocker storage
- `gcePersistentDisk` - GCE Persistent Disk
- `glusterfs` - Glusterfs volume

Типи Persistent Volumes

- `hostPath` - HostPath volume (лише для тестування одного вузла; НЕ ПРАЦЮВАТИМЕ в кластері з кількома вузлами; подумайте про використання локального обсягу замість цього)
- `iscsi` - iSCSI (SCSI over IP) storage
- `local` - локальні пристрої зберігання, встановлені на вузлах.
- `nfs` - Сховище мережевої файлової системи (NFS).
- `photonPersistentDisk` - Постійний диск Photon Controller. (Цей тип тому більше не працює після видалення відповідного хмарного постачальника.)
- `portworxVolume` - Portworx volume
- `quobyte` - Quobyte volume
- `rbd` - Rados Block Device (RBD) volume
- `scaleIO` - ScaleIO volume (deprecated)
- `storageos` - StorageOS volume
- `vsphereVolume` - vSphere VMDK volume

Persistent Volumes

Кожен PV містить специфікацію та статус, які є специфікацією та статусом тому. Ім'я об'єкта PersistentVolume має бути дійсним іменем субдомену DNS.

```
apiVersion: v1
kind: PersistentVolume
metadata:
  name: pv0003
spec:
  capacity:
    storage: 5Gi
  volumeMode: Filesystem
  accessModes:
    - ReadWriteOnce
  persistentVolumeReclaimPolicy: Recycle
  storageClassName: slow
  mountOptions:
    - hard
    - nfsvers=4.1
  nfs:
    path: /tmp
    server: 172.17.0.2
```

Persistent Volumes

Ємність

Як правило, PV матиме певну ємність для зберігання. Це встановлюється за допомогою атрибута ємності PV. Див. модель ресурсів Kubernetes, щоб зрозуміти очікувані одиниці за потужністю.

Наразі розмір пам'яті є єдиним ресурсом, який можна встановити або запросити. Майбутні атрибути можуть включати IOPS, пропускну здатність тощо.

Режим volume

СТАН ФУНКЦІЇ: Kubernetes v1.18 [стабільна]

Kubernetes підтримує два режими томів PersistentVolumes: файлова система та блок.

volumeMode є необов'язковим параметром API. Файлова система — це режим за замовчуванням, який використовується, якщо параметр volumeMode опущено.

Persistent Volumes

Режим volume

Том із `volumeMode`: файлова система монтується в Pods у каталог. Якщо том підтримується блоковим пристроєм, а пристрій порожній, Kubernetes створює файлову систему на пристрої перед першим його підключенням.

Ви можете встановити для параметра `volumeMode` значення `Block`, щоб використовувати том як необроблений блоковий пристрій. Такий том представлений у Pod як блоковий пристрій без жодної файлової системи. Цей режим корисний, щоб забезпечити Pod найшвидший можливий спосіб отримати доступ до тому без будь-якого рівня файлової системи між Pod і томом. З іншого боку, програма, що працює в Pod, повинна знати, як працювати з необробленим блоковим пристроєм.

Persistent Volumes

Режими доступу

PersistentVolume можна змонтувати на хост будь-яким способом, який підтримує постачальник ресурсів. Як показано в таблиці нижче, постачальники матимуть різні можливості, а режими доступу кожного PV налаштовані на конкретні режими, які підтримуються цим конкретним томом. Наприклад, NFS може підтримувати кілька клієнтів читання/запису, але певний NFS PV може бути експортований на сервер як доступний лише для читання. Кожен PV отримує власний набір режимів доступу, що описує можливості конкретного PV.

Режими доступу є:

- ReadWriteOnce -- том може бути змонтований як читання-запис одним вузлом
- ReadOnlyMany -- том може бути встановлений лише для читання багатьма вузлами
- ReadWriteMany -- Том може бути встановлений як читання-запис багатьма вузлами

Persistent Volumes

Режими доступу

У CSI режими доступу позначаються скорочено:

- RWO - ReadWriteOnce
- ROX - ReadOnlyMany
- RWX - ReadWriteMany

Volume Plugin	ReadWriteOnce	ReadOnlyMany	ReadWriteMany
AWSElasticBlockStore	✓	-	-
AzureFile	✓	✓	✓
AzureDisk	✓	-	-
CephFS	✓	✓	✓
Cinder	✓	-	-
CSI	depends on the driver	depends on the driver	depends on the driver
FC	✓	✓	-
FlexVolume	✓	✓	depends on the driver
Flocker	✓	-	-
GCEPersistentDisk	✓	✓	-
Glusterfs	✓	✓	✓
HostPath	✓	-	-
iSCSI	✓	✓	-
Quobyte	✓	✓	✓
NFS	✓	✓	✓
RBD	✓	✓	-
VsphereVolume	✓	-	- (works when Pods are collocated)
PortworxVolume	✓	-	✓
ScaleIO	✓	✓	-
StorageOS	✓	-	-

Persistent Volumes

Клас

PV може мати клас, який визначається встановленням атрибута `storageClassName` на ім'я `StorageClass`. PV певного класу можна прив'язати лише до PVC, які запитують цей клас. PV без `storageClassName` не має класу і може бути прив'язаний лише до PVC, які не запитують конкретний клас.

Раніше анотація `volume.beta.kubernetes.io/storage-class` використовувалася замість атрибута `storageClassName`. Ця анотація все ще працює; однак у майбутньому випуску Kubernetes він буде повністю застарілим.

Політика повернення

Поточна політика повернення:

- Retain -- відновлення вручну
- Recycle -- основний скраб (`rm -rf /thevolume/*`)
- Delete -- пов'язаний ресурс зберігання, як-от AWS EBS, GCE PD, Azure Disk або том OpenStack Cinder, видалено

Наразі переробку підтримують лише NFS і HostPath. Томи AWS EBS, GCE PD, Azure Disk і Cinder підтримують видалення.

Persistent Volumes

Параметри монтування

Адміністратор Kubernetes може вказати додаткові параметри монтування, коли постійний том монтується на вузлі. Наступні типи томів підтримують параметри підключення:

- AWSElasticBlockStore
- AzureDisk
- AzureFile
- CephFS
- Cinder (OpenStack block storage)
- GCEPersistentDisk
- Glusterfs
- NFS
- Quobyte Volumes
- RBD (Ceph Block Device)
- StorageOS
- VsphereVolume
- iSCSI

Параметри монтування не перевірені. Якщо параметр монтування недійсний, монтування не вдається.

Раніше анотація `volume.beta.kubernetes.io/mount-options` використовувалася замість атрибута `mountOptions`. Ця анотація все ще працює; однак у майбутньому випуску Kubernetes вона буде повністю застарілою.

Persistent Volumes

Спорідненість вузла

Примітка. Для більшості типів томів встановлювати це поле не потрібно. Воно автоматично заповнюється для типів блоків томів AWS EBS, GCE PD і Azure Disk. Вам потрібно явно встановити це для локальних томів.

PV може вказати спорідненість вузла, щоб визначити обмеження, які обмежують, з яких вузлів можна отримати доступ до цього тому. Блоки, які використовують PV, будуть заплановані лише для вузлів, вибраних за спорідненістю вузлів.

Фаза

Обсяг складатиметься з одного з наступних етапів:

- Доступний -- безкоштовний ресурс, який ще не прив'язаний до претензії
- Прив'язаний – том прив'язаний до претензії
- Звільнено – претензію видалено, але кластер ще не відновив ресурс
- Помилка -- тому не вдалося виконати автоматичне відновлення

CLI покаже назву PVC, прив'язану до PV.

PersistentVolumeClaims

Кожен PVC містить специфікацію та статус, які є специфікацією та статусом претензії. Ім'я об'єкта PersistentVolumeClaim має бути дійсним іменем субдомену DNS.

```
apiVersion: v1
kind: PersistentVolumeClaim
metadata:
  name: myclaim
spec:
  accessModes:
    - ReadWriteOnce
  volumeMode: Filesystem
  resources:
    requests:
      storage: 8Gi
  storageClassName: slow
  selector:
    matchLabels:
      release: "stable"
    matchExpressions:
      - {key: environment, operator: In, values: [dev]}
```

PersistentVolumeClaims

Режими доступу

Претензії використовують ті самі умовності, що й томи, коли запитують сховище з певними режимами доступу.

Режими Volume

Претензії використовують ті самі угоди, що й томи, щоб вказати використання тому як файлової системи або блокового пристрою.

Ресурси

Претензії, як і Pods, можуть запитувати певну кількість ресурсу. У цьому випадку запит на зберігання. Та сама модель ресурсів застосовується як до томів, так і до вимог.

PersistentVolumeClaims

Селектор

Претензії можуть вказати селектор міток для подальшої фільтрації набору томів. Лише томи, мітки яких відповідають селектору, можуть бути прив'язані до претензії. Селектор може складатися з двох полів:

- `matchLabels` - том повинен мати мітку з цим значенням
- `matchExpressions` - список вимог, створений шляхом визначення ключа, списку значень і оператора, який пов'язує ключ і значення. Дійсні оператори включають In, NotIn, Exists і DoesNotExist.

Усі вимоги, як від `matchLabels`, так і від `matchExpressions`, об'єднуються AND разом – усі вони мають бути задоволені, щоб відповідати.

PersistentVolumeClaims

Клас

Претензія може запитувати певний клас, вказавши назву `StorageClass` за допомогою атрибута `storageClassName`. До PVC можна прив'язати лише PV потрібного класу, ім'я класу зберігання якого збігається з PVC.

PVCs не обов'язково запитувати клас. PVC із набором `storageClassName`, що дорівнює `""`, завжди інтерпретується як такий, що запитує PV без класу, тому його можна зв'язати лише з PV без класу (без анотації або один набір, що дорівнює `""`). PVC без `StorageClassName` не зовсім однаковий і по-різному обробляється кластером залежно від того, чи ввімкнено плагін доступу `DefaultStorageClass`.

PersistentVolumeClaims

Клас

- Якщо плагін доступу ввімкнено, адміністратор може вказати StorageClass за умовчанням. Усі PVC, які не мають storageClassName, можна прив'язати лише до PV цього замовчування. Вказівка StorageClass за замовчуванням виконується встановленням значення true для анотації storageclass.kubernetes.io/is-default-class в об'єкті StorageClass. Якщо адміністратор не вказує значення за замовчуванням, кластер реагує на створення PVC так, ніби плагін доступу вимкнено. Якщо вказано більше одного за замовчуванням, плагін доступу забороняє створення всіх PVC.
- Якщо плагін доступу вимкнено, поняття StorageClass за замовчуванням не існує. Усі PVC, які не мають storageClassName, можуть бути прив'язані лише до PV, які не мають класу. У цьому випадку PVC, які не мають storageClassName, обробляються так само, як і PVC, у яких для storageClassName встановлено значення "".

PersistentVolumeClaims

Клас

Залежно від методу інсталяції, кластер `StorageClass` за замовчуванням може бути розгорнутий у кластері Kubernetes за допомогою аддон-менеджера під час інсталяції.

Коли PVC вказує селектор на додаток до запиту `StorageClass`, вимоги об'єднуються разом: лише PV запитаного класу та із запитаними мітками може бути прив'язаний до PVC.

Примітка. Наразі для PVC із непорожнім селектором не може бути динамічно створений PV.

Раніше анотація `volume.beta.kubernetes.io/storage-class` використовувалася замість атрибута `storageClassName`. Ця анотація все ще працює; однак вона не підтримуватиметься в майбутньому випуску Kubernetes.

Claims Як Volumes

Примітка про простори імен

Прив'язки PersistentVolumes є ексклюзивними, і оскільки PersistentVolumeClaims є об'єктами простору імен, підключення претензій із режимами «Багато» (ROX, RWX) можливе лише в одному просторі імен.

PersistentVolumes **ВВІВ** hostPath

PersistentVolume hostPath використовує файл або каталог на вузлі для емуляції підключеного до мережі сховища.

Підтримка Raw Block Volume

СТАН ФУНКЦІЇ: Kubernetes v1.18 [стабільна]

Наступні плагіни томів підтримують необроблені блокові томи, включаючи динамічне надання, де це можливо:

- AWSElasticBlockStore
- AzureDisk
- CSI
- FC (Fibre Channel)
- GCEPersistentDisk
- iSCSI
- Local volume
- OpenStack Cinder
- RBD (Ceph Block Device)
- VsphereVolume

Підтримка Raw Block Volume

**PersistentVolume з використанням
Raw Block Volume**

```
apiVersion: v1
kind: PersistentVolume
metadata:
  name: block-pv
spec:
  capacity:
    storage: 10Gi
  accessModes:
    - ReadWriteOnce
  volumeMode: Block
  persistentVolumeReclaimPolicy: Retain
  fc:
    targetWWNs: ["50060e801049cfd1"]
    lun: 0
    readOnly: false
```

Підтримка Raw Block Volume

PersistentVolumeClaim запитує Raw Block Volume

```
apiVersion: v1
kind: PersistentVolumeClaim
metadata:
  name: block-pvc
spec:
  accessModes:
    - ReadWriteOnce
  volumeMode: Block
  resources:
    requests:
      storage: 10Gi
```

Підтримка Raw Block Volume

Специфікація модуля додає шлях
Raw Block Device до контейнера

```
apiVersion: v1
kind: Pod
metadata:
  name: pod-with-block-volume
spec:
  containers:
    - name: fc-container
      image: fedora:26
      command: ["/bin/sh", "-c"]
      args: [ "tail -f /dev/null" ]
      volumeDevices:
        - name: data
          devicePath: /dev/xvda
  volumes:
    - name: data
      persistentVolumeClaim:
        claimName: block-pvc
```


Підтримка Raw Block Volume

Прив'язка блочних томів

Якщо користувач запитує необроблений блочний том, вказуючи це за допомогою поля volumeMode у специфікації PersistentVolumeClaim, правила зв'язування дещо відрізняються від попередніх випусків, у яких цей режим не вважався частиною специфікації. У списку наведено таблицю можливих комбінацій, які користувач і адміністратор можуть вказати для запиту необробленого блокового пристрою. Таблиця вказує, чи буде зв'язаний том, чи не надано комбінації: Матриця зв'язування томів для статично наданих томів:

PV volumeMode	PVC volumeMode	Result
unspecified	unspecified	BIND
unspecified	Block	NO BIND
unspecified	Filesystem	BIND
Block	unspecified	NO BIND
Block	Block	BIND
Block	Filesystem	NO BIND
Filesystem	Filesystem	BIND
Filesystem	Block	NO BIND
Filesystem	unspecified	BIND

Знімок томів і відновлення томів із підтримки знімків

СТАН ФУНКЦІЇ: Kubernetes v1.20 [стабільна]

Знімки томів підтримують лише плагіни томів CSI поза деревом. Плагіни томів у дереві більше не підтримуються.

Створіть PersistentVolumeClaim із знімка тома

```
apiVersion: v1
kind: PersistentVolumeClaim
metadata:
  name: restore-pvc
spec:
  storageClassName: csi-hostpath-sc
  dataSource:
    name: new-snapshot-test
    kind: VolumeSnapshot
    apiGroup: snapshot.storage.k8s.io
  accessModes:
    - ReadWriteOnce
  resources:
    requests:
      storage: 10Gi
```

Клонування томів

Клонування томів доступне лише для плагінів томів CSI.

Створіть PersistentVolumeClaim з існуючого PVC

```
apiVersion: v1
kind: PersistentVolumeClaim
metadata:
  name: cloned-pvc
spec:
  storageClassName: my-csi-plugin
  dataSource:
    name: existing-src-pvc-name
    kind: PersistentVolumeClaim
  accessModes:
    - ReadWriteOnce
  resources:
    requests:
      storage: 10Gi
```

Написання портативної конфігурації

Якщо ви пишете шаблони конфігурації або приклади, які працюють на широкому діапазоні кластерів і потребують постійного сховища, рекомендується використовувати такий шаблон:

- Включіть об'єкти `PersistentVolumeClaim` у свій пакет конфігурації (поряд із розгортаннями, `ConfigMaps` тощо).
- Не включайте об'єкти `PersistentVolume` до конфігурації, оскільки користувач, який створює екземпляр конфігурації, може не мати дозволу на створення `PersistentVolumes`.
- Дайте користувачеві можливість надати назву класу зберігання під час створення екземпляра шаблону.
 - Якщо користувач надає назву класу зберігання, введіть це значення в поле `persistentVolumeClaim.storageClassName`. Це змусить PVC відповідати правильному класу зберігання, якщо адміністратор увімкнув для кластера `StorageClasses`.
 - Якщо користувач не вказує назву класу зберігання, залиште поле `persistentVolumeClaim.storageClassName` нульовим. Це спричинить автоматичне надання PV для користувача з кластером `StorageClass` за умовчанням у кластері. У багатьох кластерних середовищах встановлено `StorageClass` за замовчуванням, або адміністратори можуть створити власний `StorageClass` за замовчуванням.
- У ваших інструментах слідкуйте за PVC, які не зв'язуються через деякий час, і покажіть це користувачеві, оскільки це може означати, що кластер не має підтримки динамічного зберігання (у цьому випадку користувач повинен створити відповідний PV) або кластер не має системи зберігання (у цьому випадку користувач не може розгорнути конфігурацію, яка вимагає PVC).

Питання та відповіді